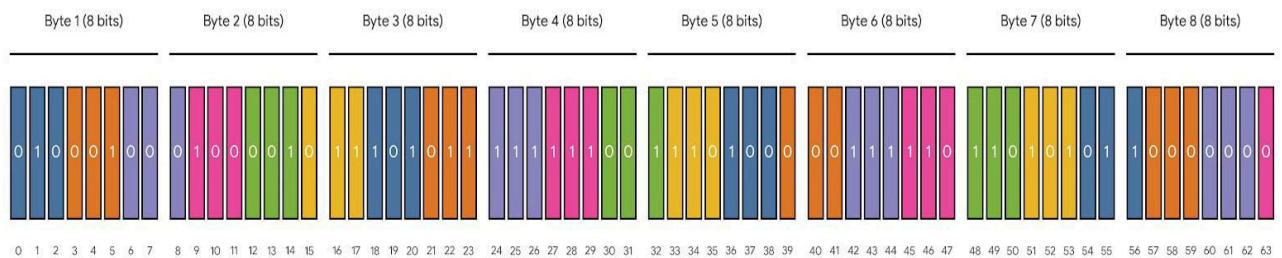


Alejandro Escobar Arbelaez 1023629917

Pablo Colorado Correa 1018235473

Este desafío consiste en diseñar e implementar en C++ (con framework Qt) el juego Sweet Crush, donde el tablero de dimensiones dinámicas debe almacenarse obligatoriamente de forma compacta como una secuencia continua de bits empaquetada en memoria dinámica (unsigned char* en nuestro caso) utilizando exactamente 3 bits por posición (lo que requiere el uso riguroso de operadores bitwise como &, |, << y >> para leer, modificar o dividir fichas entre bytes). Cada ficha está representada de la siguiente forma: 000 = Y, 001 = A, 010 = K, 011 = M, 100 = Q, 101 = R, 110 = VACIO y 111 = INVALIDO. A continuación se presenta una gráfica que representa el manejo de los bits.



La solución debe gestionar la eliminación de fichas, detección de combinaciones horizontales y verticales, caídas de fichas y combinaciones en cadena (cascadas), así como la modificación estructural del tablero mediante la inserción o eliminación de filas y columnas en cualquier posición

Planteamiento:

1. Estructura de datos:

la estructura de datos seleccionada fue un arreglo dinámico por medio de un unsigned char*.

Se optó por esta estructura lineal por la dependencia de ciertas fichas a la continuidad de los bits más allá de las variables, por el cual el deseo de separar el arreglo en más dimensiones para su posible mejor manejo, sería ineficiente y contraproducente, pues para la reacomodación de los elementos de una fila, se tendría que asegurar también la reacomodación de todos los elementos anteriores a esa fila (en el caso de que quisiéramos hacer una estructura matricial donde cada fila en un arreglo distinto).

Se llegó a considerar la opción de manejar la estructura de datos por medio de un puntero a punteros, pero se descartó por lo previamente planteado.

Se acepta que el riesgo de un costo computacional adicional para insertar nuevos elementos en medio de la estructura, pero al tener claro que en el flujo de juego del usuario común, las operaciones de redimensionar el tablero no serán de uso recurrente, esto, en todo caso, será un trade-off beneficioso

Para la creación de la cadena, se le pedía al usuario al inicio de su sesión el tamaño inicial del tablero, con lo cual se podrá calcular el número de bytes que han de ser reservados para almacenar la entidad de las fichas (más un margen de espacio útil vacío para facilitar operaciones de modificación), esto se hará por medio de la operación $(segment_size * rows * cols / 8)$, más un byte más.

los índices se pueden manejar de manera totalmente aritmética, de esta forma

$indice = fila * (ancho\ de\ fila\ o\ número\ de\ columnas) + columna$

esta fórmula funciona de la siguiente manera: cada fila está compuesta por n elementos, por lo cual el índice inicia diciendo, saltamos x filas, o sea $(x * filas * n)$ elementos, si queremos acceder a un dato en la fila 2, aprovechando que los índices comienzan desde cero, escribimos $(2 * n) + indice_columna$.

este índice se deberá multiplicar por 3 para obedecer el orden a la escalada de segmentos por 3 bits, o sea, el índice pasaría a medir el número de fichas posteriores a la posición actual a el número de bits anteriores a la posición actual, este será el verdadero punto de inicio de la posición, o sea, en que bit estamos ubicados.

igualmente, este sistema de índices es insuficiente por sí mismo, ya que, al ser las fichas representadas por segmentos de 3 bits, número no múltiplo al número de bits en un byte, a lo largo que se va llenando la cadena, se va a ir desfasando, por lo cual, se ha de tener que calcular ese desfase, como? viendo el desfase acumulado que se deja en cada variable, por ejemplo, en el primer byte: se ocuparan 6 bits con sus fichas enteramente propias al mismo byte, y quedarían 2 bits residuos, que harán parte de la siguiente ficha compartida con el siguiente byte, al hablar de bits residuos, es lógico pensar que a partir de una operación módulo, se podrá calcular este offset, siendo tal vez, $(indice * 3 \% 8)$ la operación que nos indicará el offset, o, cuantos bits quedan sueltos si divides los bits en donde estás parado en bloques de 8.

una manera muy eficiente de evitar tener que reacomodar básicamente la entidad del arreglo a ciertos cambios, se utilizara la representación 111 para invalido, de modo en que si se borra una fila o columna, no se tendrá que desplazar nada, solo se cambiará el valor de los bits de dichas posiciones al invalido para simbolizar que ya no están en uso, para marcar cuántas casillas están en desuso, se va a utilizar una variable de conteo, si la variable de conteo marca que aproximadamente el 35% de las casillas de fichas estan en desuso, se hara una reacomodacion de la estructura de datos.

para la reasignación, no hay alternativa mejor, aunque ineficiente, que copiar y pegar todo el arreglo en uno de menor tamaño, al igual que si se expande, es el precio a pagar

¿Cómo vamos a saber si se dio un patrón válido de 3 fichas? se buscará solo si se dio un cambio en lo que está arriba, y los alrededores cercanos de la posición de la ficha eliminada, se hará un bucle de sentido reduciendo poco a poco el número de filas en la operación previamente indicada para el cálculo del índice global.

Como desde la estructura de datos planteada, se permite cierta cantidad de espacio extra, este espacio será el utilizado a la hora de hacer inserciones... a menos de que ya no haya de otra, en ese caso, se tendrá que pedir un nuevo espacio de memoria.

para operar los bits dentro de las variables, se utilizarán desplazamientos bitwise y mascarar